

Examen réparti 2

Master 1 SAR

UE 4I401

L. Arantes, S. Dubois, J. Sopena, P. Sens

Janvier 2023

2 heures – **Aucun document autorisé**

Tout appareil électronique interdit

Le barème sur 20,5 points est donné à titre indicatif

Exercice 1 – Questions de cours (1,5 points)

Question 1 $\frac{1}{2}$ point

La traduction d'une adresse virtuelle en adresse physique est-elle réalisée par le matériel ou par le système ?

Solution:

Matériel

Question 2 $\frac{1}{2}$ point

Que contient le bloc 0 d'une partition ? Quel est son rôle ?

Solution:

Le code de chargement du système (plus la table des partitions). Son rôle est de charger le système d'exploitation.

Question 3 $\frac{1}{2}$ point

L'inode d'un même fichier peut-elle être à un même moment plusieurs fois en mémoire dans la table inode[] ? Justifiez votre réponse.

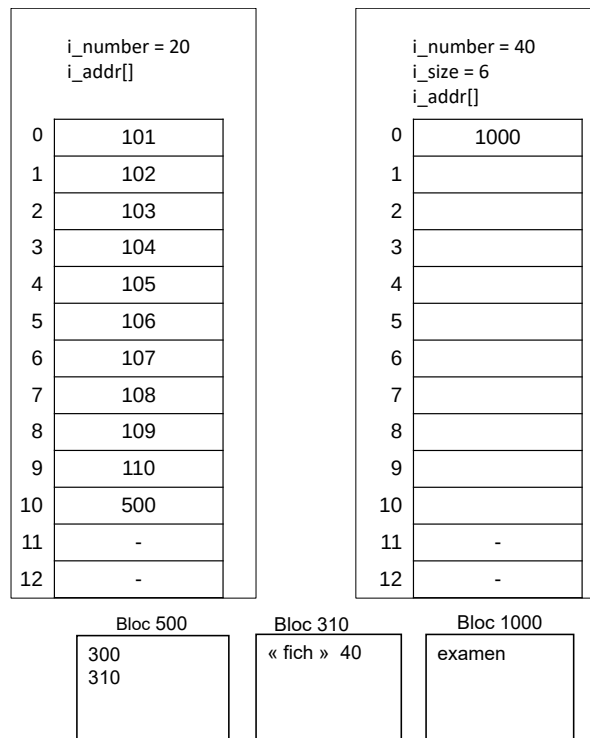
Solution:

Non en cas d'utilisation multiple seule le `i_count` est incrémenté.

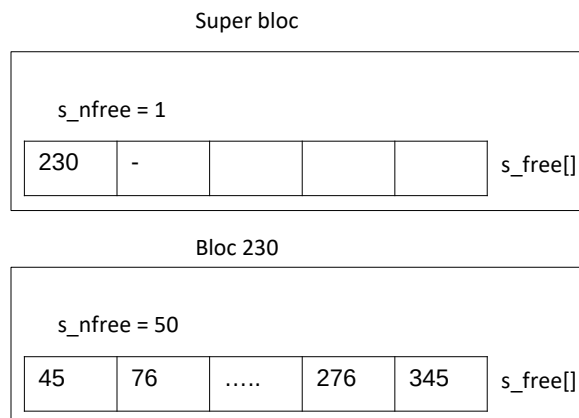
Exercice 2 – Système de fichiers (5,5 points)

Les *inodes* ont une structure équivalente à celle vue en TD. Les inodes sont numérotées à partir de 1, ont une taille de 32 octets. Les blocs disque ont une taille de 512 octets.

Le répertoire courant (`u.u_cdir`) correspond à l'inode numéro 20. Nous considérons les *inodes* et les blocs suivants :



Les blocs 101 à 110, ainsi que le bloc 300 ne contiennent pas la chaîne de caractères "fich".
Le superbloc a initialement la configuration suivante :



Question 1 $\frac{1}{2}$ point

Quels sont les numéros de blocs retournés par les appels à `bmap(u.u_cdir, 0, 0)` et `bmap(u.u_cdir, 10, 0)` ?

Solution:

blocs 101 et 300

Soit le code utilisateur suivant :

```
1  int fd1,fd2,fd3;
2  char buffer[16];
3  fd1 = open("./fich", O_RDWR);
4  lseek(fd1, 512, SEEK_SET);
5  write(fd1, "EXA", 3);
6  if(fork()==0) {
7      write(fd1,"MEN", 3);
8      fd2 = open("./fich", O_RDONLY);
9      if (fork()==0) {
10         read(fd2, buf, 3);
11         buf[3] = 0;
12         fd3 = open("./fich", O_RDWR);
13         printf("%s\n", buf);
14         close(fd1); close(fd2); close(fd3);
15         exit(0);
16     }
17     else {
18         close(fd2);
19         wait(NULL);
20         close(fd1);
21     }
22 }
23 wait(NULL) ;
24 close(fd1);
25 exit(0);
```

Question 2 $\frac{1}{2}$ point

Quel est l' affichage produit par le printf de la ligne 13 (justifiez) ?

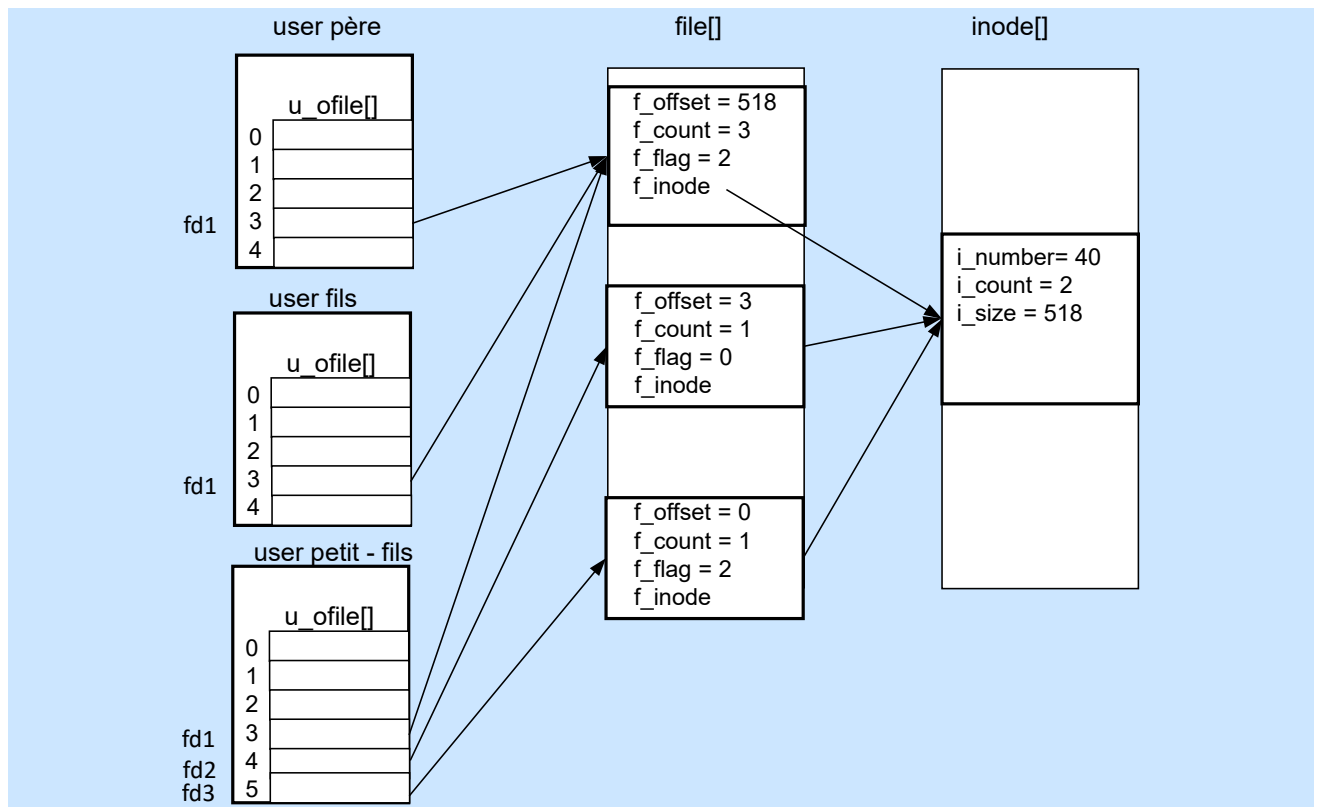
Solution:

Ligne 13 : "exa"

Question 3 $1\frac{1}{2}$ points

En supposant que la ligne 19 a été exécutée, donnez l'état juste après la ligne 13 des tables de descripteurs de fichiers, fichiers ouverts, et inodes (faites un schéma, pensez à représenter notamment les champs *f_count*, *f_offset*, *f_flags*, *i_count*, *i_size* et *i_number*).

Solution:



Question 4 2 points

Avant l'exécution de ce code, nous faisons l'hypothèse que seule inode du répertoire courant (inode numéro 20) est présente dans la table `inode[]`. Donnez la séquence des blocs et des inodes accédés pour les lignes 3 et 4. Pour chaque accès, spécifiez la ligne de code associée.

Solution:

3- open -namei
inode 20 en mémoire
accès bloc direct répertoire 101 à 110
accès bloc indirection 500
accès bloc répertoire 300 et 310
accès bloc inode 4 (= 2 + (40-1)/16)
accès inode 40
4- lseek - aucun accès

Question 5 1½ points

Quels sont les blocs accédés par la ligne 5. Donnez l'état du superbloc juste après la ligne 5.

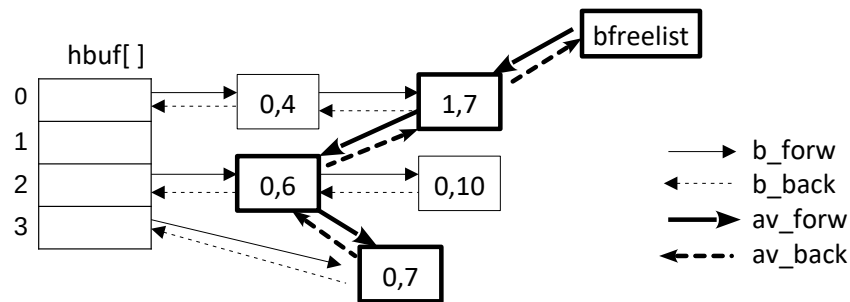
Solution:

Accès en écriture à un nouveau bloc => allocation/accès bloc 230
Le SB devient vide, le contenu de 280 est copié dans le SB

- s_nfree = 50
- sfree = [45, 76, ..., 276, 345]

Exercice 3 – Buffer cache (4 points)

Nous considérons un buffer cache de **5 cases**. La figure suivante donne l'état du buffer cache. Dans chaque buffer est indiqué le doublet $\langle dev, bloc \rangle$ qui correspond au numéro de device et de bloc associé au buffer. On suppose qu'aucun buffer n'a initialement le flag `DEL_WRI`.



Question 1 $\frac{1}{2}$ point

Dans cette configuration, quelle est la fonction de hachage $fh(dev, bloc)$ qui permet de répartir les blocs dans hbuf ?

Solution:

$$fh(dev, bloc) = (dev + bloc) \% 4$$

Question 2 1 point

Quels sont les buffers dont les entrées/sorties sont terminées ? Dans quel ordre ont eu lieu ces entrées/sorties.

Solution:

Blocs $\langle 1, 7 \rangle$, $\langle 0, 6 \rangle$ et $\langle 0, 7 \rangle$ avec les E/S faites dans cet ordre.

Question 3 $1\frac{1}{2}$ points

Le noyau exécute la séquence suivante :

```
1 struct buf *bp;
2 bp = bread(1,7);
3 brelse(bp);
4 bp=bread(1,3);
5 brelse(bp);
6 bp = bdwrite(0,7);
```

Donnez l'état de la bfreelist (la liste des blocs qui la compose de la tête vers la queue) après les lignes 3, 5 et 6 ainsi que le nombre d'entrées/sorties physiques générées par cette séquence (les codes de brelse, bread et bdwrite sont rappelés en annexe).

Solution:

Init : $\langle 1, 7 \rangle \langle 0, 6 \rangle \langle 0, 7 \rangle$

ligne 3 $\langle 1, 7 \rangle$: $\langle 0, 6 \rangle \langle 0, 7 \rangle$, $\langle 1, 7 \rangle$

ligne 5 $\langle 1, 3 \rangle$: $\langle 0, 7 \rangle$, $\langle 1, 7 \rangle \langle 1, 3 \rangle$, 1 E/S

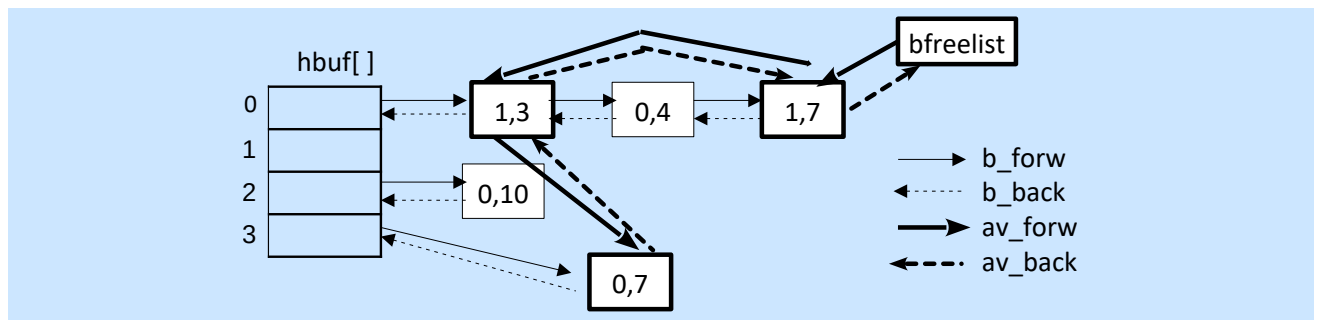
ligne 6 $\langle 0, 7 \rangle$: $\langle 1, 7 \rangle \langle 1, 3 \rangle \langle 0, 7 \rangle$

Total : 1 E/S

Question 4 1 point

Faites un schéma complet du buffer cache (hbuf, chainage des buffers et de la bfreelist) après la ligne 6.

Solution:



Exercice 4 – Swap (4,5 points)

On considère le swapper du TD dont le code est rappelé en annexe.

Question 1 1 point

Rappelez le rôle des variables `runin` et `runout`.

Solution:

Ce sont deux variables globales sur lequel le swapper s'endort (`runout++`; `sleep(&runout)` et `runin++`; `sleep(&runin)`)

Il s'endort sur la variable :

- `runout` lorsqu'il n'a pas trouvé de processus à ramener en mémoire (un processus SRUN et non chargé)
- `runin` lorsqu'il n'a pas trouvé de processus en mémoire éligible à être swapper

Question 2 1/2 point

Donnez une raison pour faire un `wakeup` sur `runout`

Solution:

Un `wakeup` sur `runout` se fait :

- lorsqu'un processus passe à l'état RUN mais n'est pas chargé en mémoire (fonction `setrun`)
- lors d'un autoswap (fonction `xswap`)

Question 3 1/2 point

Quelles fonctions appellent le `wakeup` sur `runin`

Solution:

Un `wakeup` sur `runin` se fait :

- toutes les secondes (fonction `realtime`)
- lorsqu'un processus s'endort (fonction `sleep`)
- lorsqu'un processus se termine (fonction `exit` appelle `mfree` qui fait le `wakeup`)

On considère que la mémoire comporte **8 unités** et la table `proc` ne comporte que 5 processus. Voici l'état de cette table juste après l'exécution de la fonction `realtime` où `p_size` est exprimé en nombre d'unités :

p_pid	p_flag	p_stat	p_size	p_time	p_wchan	p_pri
0	SSYS SLOAD	SSLEEP	2	120	&runout	PSWP
1	SSYS SLOAD	SRUN	2	115		PZERO - 1
100	SLOAD	SSLEEP	2	1	&cond	PZERO + 1
101	SLOAD	SSLEEP	1	2	&cond	PZERO + 1
102	SLOAD	SRUN	1	1		
103	0	SSLEEP	4	1	&cond2	PZERO + 1

On fait les hypothèses suivantes :

- Aucun `fork` ni `exit`, ni `wakeup(&cond)` ne sera appelé par la suite.
- Le temps d'attente de fin d'entrée/sortie est considéré négligeable par rapport à une seconde.
- On ne prendra pas en compte les segments texte des processus.

Question 4 2½ points

Le processus 103 passe à l'état **SRUN**, donnez l'évolution de la table jusqu'à ce qu'il soit chargé en mémoire. Si besoin vous pouvez considérer un ou plusieurs appels à `realtime`. Pour simplifier, vous pouvez lister uniquement la liste des champs qui changent.

Solution:

Le passage à l'état **SRUN** entraîne :

1. switch vers le swapper immédiatement car il est endormi sur `runout` (`wakeup` dans le `sleep`) et qu'il a la plus forte priorité.
2. le swapper choisit 103 pour le sortir du swap car ce dernier est **SRUN** mais pas **SLOAD**
3. impossible de charger 103 car la mémoire est pleine, il choisit donc de swapper 100 puis 101 car ils sont l'état `sleep` à une priorité interruptible ($\Rightarrow$ I/O pour swapper les 2 processus).
4. une fois l'I/O terminée, le swapper se réveille dans `xswap` : **on enlève le SLOAD de 100, puis 101, leur p_time passe à 0**
5. il rechoisit 103 $\rightarrow$ toujours pas de place en mémoire et aucun processus à évincer (tous **SSYS** ou bien avec à l'état **SRUN**), 103 étant depuis moins de 3 sec. sur le swap, **le swapper s'endort donc sur runin**
6. après 2 sec., 103 est depuis 3 sec. en mémoire, cette fois ci 102 peut être choisi même s'il a l'état **SRUN** car il est en mémoire depuis plus de 2 secondes.
7. 102 est déchargé, les `p_time` de 102 et 103 passe à 0.

On a donc au final :

p_pid	p_flag	p_stat	p_size	p_time	p_wchan	p_pri
0	SSYS SLOAD	SSLEEP	2	122	&runin	PSWP
1	SSYS SLOAD	SRUN	2	117		PZERO - 1
100		SSLEEP	2	2	&cond	PZERO + 1
101		SSLEEP	1	2	&cond	PZERO + 1
102		SRUN	1	0		PZERO + 1
103	SLOAD	SRUN	4	0		

Exercice 5 – Synchronisation et appel système (5 points)

On souhaite écrire un nouvel appel système `int prefetch(int fd, int nblocs)`. Cet appel permet de charger dans le buffer cache `nblocs` blocs logiques consécutifs à partir de l'offset du fichier `fd` préalablement ouvert en lecture. Par exemple, si l'offset courant de `fd` (`f_offset`) est égal à 1600, l'appel à `prefetch(fd, 5)` chargera dans la bfreelist les blocs logiques 3 ($=1600/512$), 4, 5, 6 et 7 du fichier. L'appel retourne le nombre de blocs effectivement pré-chargés : seuls les blocs alloués du fichier peuvent être préchargés (dont l'entrée n'est pas égale à 0 dans `i_addr`), de même le pré-chargement s'arrête si on a atteint la fin du fichier.

Question 1 3 points

Donnez le code dans le noyau de l'appel système `sys_prefetch()` correspondant à `int prefetch(int fd, int nblocs)`.

Solution:

```
sys_prefetch () {
    int fd=u.u_arg[0];
    int nblocs=u.u_arg[1];
    struct inode *ip=u_ofile[fd]->f_inode;
    daddr_t db;
    struct buf *bp;
    int count = 0;
    int first = u_ofile[fd]->f_offset/BSIZE
    int last = ( first+nblocs-1 < ip->isize/BSIZE )? first+nblocs-1 : ip
        ->i_size/BSIZE;

    for (i = first; i <= last; i++) {
        if ( (db = bmap(ip,i,B_READ)) == -1)
            continue;
        bp= bread(ip->i_dev, db);
        brelse(bp);
        count++;
    }
    u.u_ar[R0] = count;
}
```

Question 2 1½ points

On souhaite modifier la fonction `sys_prefetch()` pour ne pré-charger uniquement les blocs que s'il y a suffisamment de place dans la bfreelist (champs `bfreelist.b_bcount`) avant le chargement du premier bloc. Pour simplifier on ne prendra pas en compte le chargement d'éventuels blocs d'indirection et on supposera que le nombre de blocs à précharger est inférieur à la capacité totale du buffer cache.

S'il n'y a pas assez de buffers disponibles, l'appel doit se bloquer à une priorité non interruptible en appelant `sleep`.

Solution:

```
sys_prefetch () {
    ...
    while (bfreelist.b_bcount < last-first+1)
        sleep((caddr_t) &bfreelist+1, PZERO -1);
    ...
}
```

Question 3 ½ point

Quelle fonction interne devra faire le wakeup pour réveiller un processus bloqué dans `sys_prefetch()` ?

Solution:

brelse

Annexe

```
#define NADDR 13

struct inode
{
    char        i_flag;
    char        i_count; /* reference count */
    dev_t       i_dev;   /* device where inode resides */
    ino_t       i_number; /* i number, 1-to-1 with device address */
    unsigned short i_mode;
    short       i_nlink; /* directory entries */
    short       i_uid;   /* owner */
    short       i_gid;   /* group of owner */
    off_t       i_size;  /* size of file */
    struct {
        daddr_t    i_addr[NADDR]; /* if normal file/directory */
        daddr_t    i_lastr;        /* last logical block read (for read-
                                   ahead) */
    };
};

struct inode inode[NINODE]; /* The inode table itself */

/*
 * One file structure is allocated
 * for each open/creat/pipe call.
 */
struct file
{
    char        f_flag;
    cnt_t       f_count; /* reference count */
    int         *f_inode; /* pointer to inode structure */
    long        f_offset; /* read/write character index */
} file[NFILE];

/* flags */
#define FOPEN      (-1)
#define FREAD      0x0001
#define FWRITE     0x0002
#define FPIPE      0x0004

/* open parameters */
#define O_RDONLY   0
#define O_WRONLY   1
#define O_RDWR     2
```

```

/*
 * The main loop of the scheduling (swapping) process.
 * The basic idea is:
 *   see if anyone wants to be swapped in;
 *   swap out processes until there is room;
 *   swap him in;
 *   repeat.
 * The runout flag is set whenever someone is swapped out.
 * Sched sleeps on it awaiting work.
 *
 * Sched sleeps on runin whenever it cannot find enough
 * memory (by swapping out or otherwise) to fit the
 * selected swapped process. It is awakened when the
 * memory situation changes and in any case once per second.
 */

sched()
{
    register struct proc *rp, *p;
    register struct text *xp;
    register int a, n;

    /*
     * find user to swap in;
     * of users ready, select one out longest
     */

loop:
    spl(CLINHB);
    n = -1;
    for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
        if (rp->p_stat==SRUN && (rp->p_flag&SLOAD) == 0 &&
            rp->p_time > n) {
            p = rp;
            n = rp->p_time;
        }

    /*
     * If there is no one there, wait.
     */
    if (n == -1) {
        runout++;
        sleep((caddr_t)&runout, PSWP);
        goto loop;
    }
    spl(NORMAL);

    /*
     * See if there is memory for that process;
     */

    rp = p;
    a = rp->p_size;
    if((xp=rp->p_textp) != NULL)
        if(xp->x_ccount == 0)
            a += xp->x_size;
    if((a=malloc(coremap, a)) != NULL)
        goto found2;

    /*

```

```

    * none found.
    * look around for easy memory.
    * Select the largest of those sleeping
    * at bad priority; if none, select the oldest.
    */

spl(CLINHB);
for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
    if ((rp->p_flag&(SSYS|SLOCK|SLOAD))==SLOAD &&
        ( (rp->p_stat==SSLEEP && rp->p_pri >= PZERO) ||
          rp->p_stat==SSTOP))
        goto found1;
if(n < 3)
    goto sloop;
n = -1;
for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
    if ((rp->p_flag&(SSYS|SLOCK|SLOAD))==SLOAD &&
        (rp->p_stat==SRUN || rp->p_stat==SSLEEP) &&
        rp->p_time > n) {
        p = rp;
        n = rp->p_time;
    }
if(n < 2)
    goto sloop;
rp = p;

/*
 * swap user out
 */
found1:
spl(NORMAL);
rp->p_flag &= ~SLOAD;
xswap(rp, 1, 0);
goto loop;

/*
 * swap user in
 */
found2:
rp=p;
if((xp=rp->p_textp) != NULL) {
    if(xp->x_ccount == 0) {
        if(swap(xp->x_daddr, a, xp->x_size, B_READ))
            goto swaper;
        xp->x_caddr = a;
        a += xp->x_size;
    }
    xp->x_ccount++;
}
if(swap(rp->p_addr, a, rp->p_size, B_READ))
    goto swaper;
mfree(swapmap, (rp->p_size+7)/8, rp->p_addr);
rp->p_addr = a;
rp->p_flag |= SLOAD;
rp->p_time = 0;
goto loop;

sloop:
runin++;
sleep((caddr_t)&runin, PSWP);

```

```
        goto loop;

swaper:
    panic("swap_error");
}
```

```

daddr_t
bmap(ip, bn, rwflg)
    struct inode *ip;
daddr_t bn;
int rwflg;
{
    ...
}

/*
 * Read in (if necessary) the block and return a buffer pointer.
 */
struct buf *
bread(dev, blkno)
    dev_t dev;
daddr_t blkno;
{
    register struct buf *bp;

    bp = getblk(dev, blkno);
    if (bp->b_flags & B_DONE)
        return(bp);
    bp->b_flags |= B_READ;
    bp->b_bcount = BSIZE;
    (*bdevsw[bmajor(dev)].d_strategy)(bp);
    iowait(bp);
    return(bp);
}

/* Release the buffer, marking it so that if it is grabbed
 * for another purpose it will be written out before being
 * given up (e.g. when writing a partial block where it is
 * assumed that another write for the same block will soon follow).
 * This can't be done for magtape, since writes must be done
 * in the same order as requested.
 */
bdwrite(bp)
register struct buf *bp;
{
    bp->b_flags |= B_DELWRI | B_DONE;
    bp->b_resid = 0;
    brelse(bp);
}

/*
 * release the buffer, with no I/O implied.
 */
brelse(bp)
register struct buf *bp;
{
    register struct buf **backp;
    register s;

    if (bp->b_flags & B_WANTED)

```

```

        wakeup((caddr_t)bp);
if (bfreelist.b_flags&B_WANTED) {
    bfreelist.b_flags &= ~B_WANTED;
    wakeup((caddr_t)&bfreelist);
}
if (bp->b_flags&B_ERROR) {
    bp->b_flags &= ~(B_ERROR|B_DELWRI);
    bp->b_error = 0;
}

```

```

s = gpl();
spl(BDINHB);
backp = &bfreelist.av_back;
(*backp)->av_forw = bp;
bp->av_back = *backp;
*backp = bp;
bp->av_forw = &bfreelist;
bp->b_flags &= ~(B_WANTED|B_BUSY|B_ASYNC);
bfreelist.b_bcount++;
spl(s);
}

```